

实验 4：运算部件设计实验报告

姓名： 学号：

2026 年 4 月 20 日

1 实验目的

1. 掌握先行进位部件 CLU 和先行进位加法器 CLA 的设计方法。
2. 掌握 32 位快速加法器和标志位的设计方法。
3. 掌握 32 位桶形移位器的设计方法。
4. 掌握 RV32I 算术逻辑部件的设计方法。

2 实验环境

- 软件环境：Logisim
- 实验平台：数字逻辑与计算机基础实验平台

3 实验内容与结果分析

3.1 Lab4.1：4 位先行进位加法器 CLA

3.1.1 实验原理

为了提高加法器的运算速度，需要尽量减少进位逐级传递带来的延迟。先行进位加法器的核心思想是先分别计算每一位的进位传递函数 P_i 和进位生成函数 G_i ，再直接由输入和低位进位 C_0 并行推出各级进位信号。

定义：

$$P_i = X_i + Y_i, \quad G_i = X_i Y_i$$

则 4 位先行进位部件可以直接得到：

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1G_0 + P_1P_0C_0$$

$$C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$$

$$C_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0$$

为了支持更高位数扩展，还需要生成组传递信号 Pg 和组产生信号 Gg :

$$Pg = P_3P_2P_1P_0$$

$$Gg = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0$$

在此基础上，再结合 4 个支持 P 、 G 输出的全加器，即可实现 4 位快速加法器 CLA。

3.1.2 4 位 CLU 电路设计

首先构建 4 位先行进位部件 CLU，实现 $C_1 \sim C_4$ 以及 Pg 、 Gg 的组合逻辑输出。

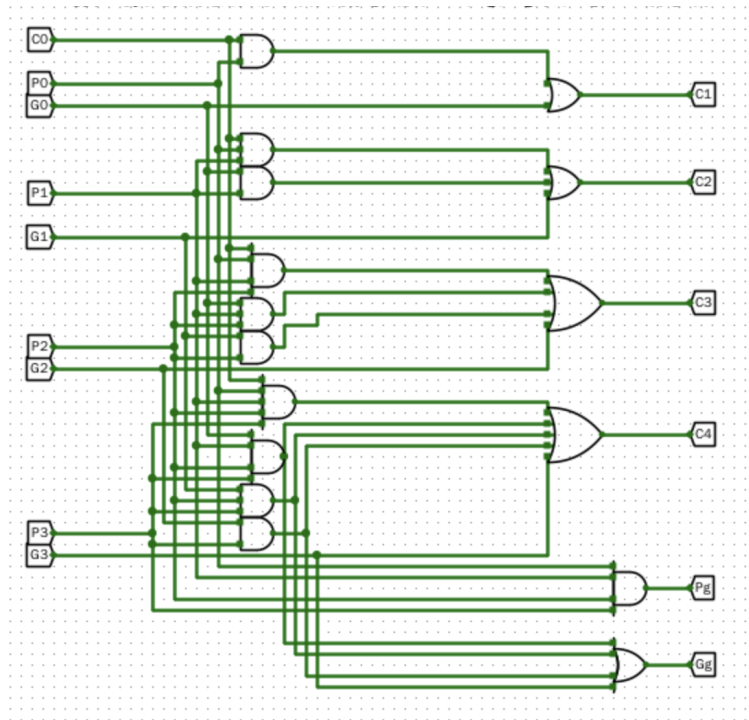


图 1: 4 位先行进位部件 CLU 电路图

3.1.3 4 位 CLU 仿真测试

对不同的 P_i 、 G_i 和 C_0 进行赋值，观察各级进位以及组信号输出，验证 CLU 电路逻辑正确。

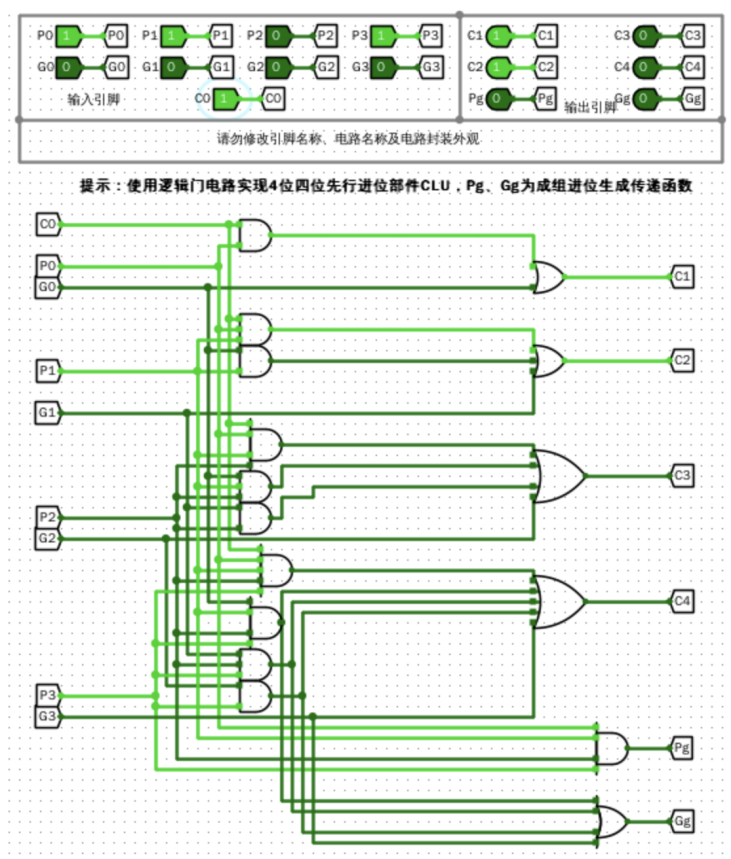


图 2: 4 位先行进位部件 CLU 仿真测试图

3.1.4 支持 P 、 G 的全加器设计

在 CLA 中，全加器除了输出求和结果外，还需要输出本位的进位传递函数 P 和进位生成函数 G 。其逻辑表达式为：

$$F = A \oplus B \oplus Cin$$

$$P = A + B$$

$$G = A \cdot B$$

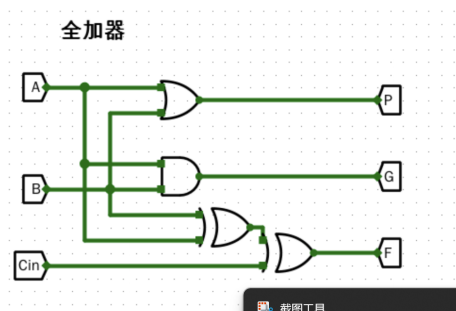


图 3: 支持进位生成与传递函数的全加器电路图

3.1.5 全加器仿真测试

对输入 A 、 B 、 Cin 进行测试，观察和位 F 、传递信号 P 、生成信号 G 的输出是否符合理论分析。

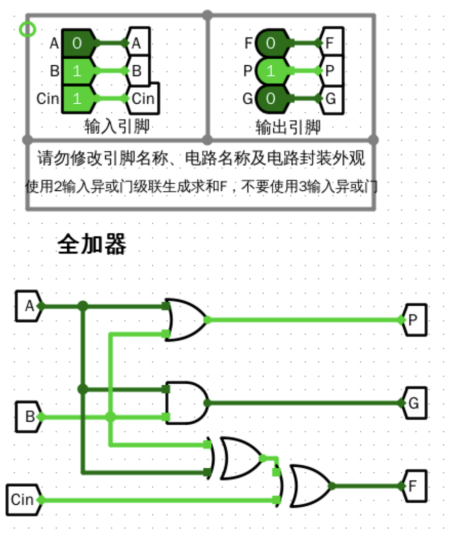


图 4: 全加器仿真测试图

3.1.6 4 位快速加法器 CLA 设计

将 1 个 CLU 和 4 个支持 P 、 G 的全加器组合，构成 4 位快速加法器。该加法器不仅能输出和 $F[3:0]$ 与最高位进位 $Cout$ ，还可输出组间进位传递信号 Pg 和组间进位生成信号 Gg ，为更高位扩展做准备。

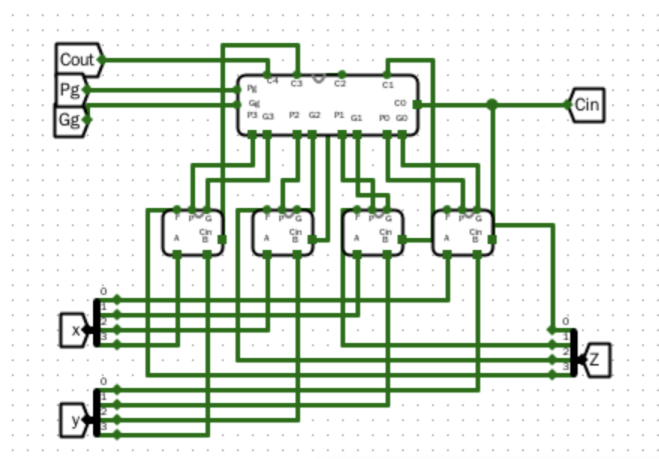


图 5: 4 位快速加法器 CLA 电路图

3.1.7 4 位快速加法器仿真测试

设置不同的输入 X 、 Y 和 Cin ，对求和结果、进位输出和组信号进行测试。实验结果表明，4 位 CLA 能够正确实现快速加法，并且输出的 Pg 、 Gg 符合分组先行进位

扩展需求。

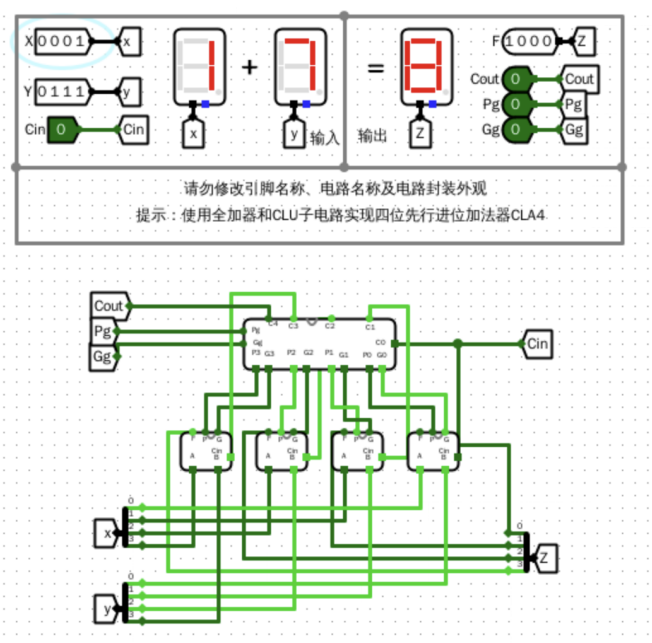


图 6: 4 位快速加法器 CLA 仿真测试图

3.2 Lab4.2: 16 位两级先行进位加法器

3.2.1 实验原理

16 位加法器可分成 4 个 4 位分组，每组使用一个 4 位快速加法器 CLA。组间的进位则通过更高一级的先行进位部件统一生成，从而实现两级先行进位结构。

这种方法相较于普通串行进位加法器，大大减小了关键路径上的进位传递延迟。其设计思想为：

- 组内由 4 位 CLA 负责并行计算位间进位；
- 组间由 CLU 根据各组的 Pg 和 Gg 计算更高层进位；
- 最终实现 16 位加法的快速求解。

3.2.2 电路设计

在主电路中调用 4 个 4 位 CLA 子电路和 1 个组间 CLU 子电路，构成 16 位两级先行进位加法器。

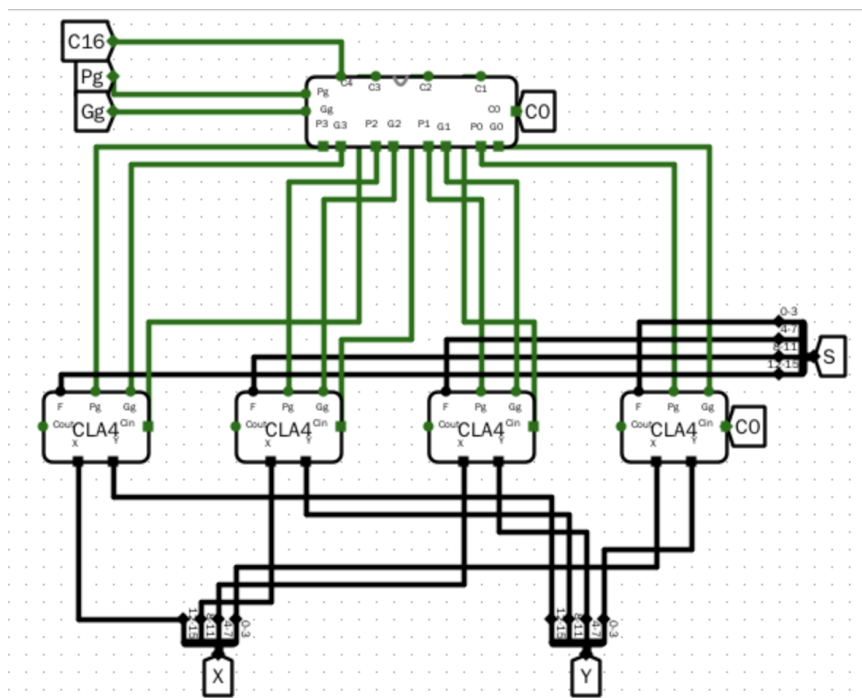


图 7: 16 位两级先行进位加法器电路图

3.2.3 仿真测试与分析

通过改变 16 位输入 X 、 Y 和低位进位 C_{in} ，观察结果输出 F 、最高位进位 C_{out} 及扩展输出信号，验证电路逻辑正确。实验结果表明，分组先行进位结构能够正确实现 16 位快速加法运算。

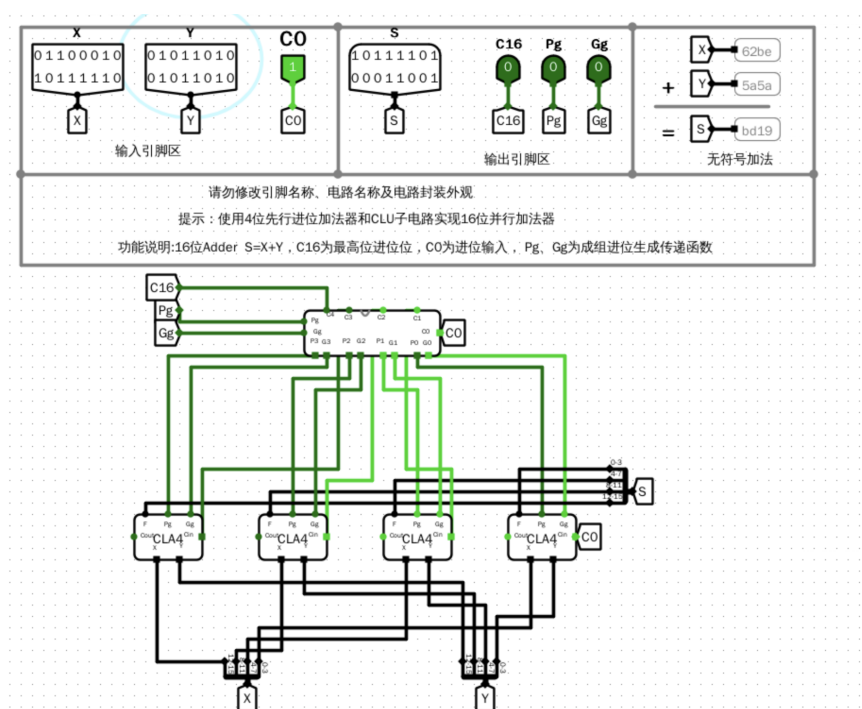


图 8: 16 位两级先行进位加法器仿真测试图

3.3 Lab4.3: 32 位快速加法器

3.3.1 实验原理

32 位快速加法器由两个 16 位两级先行进位加法器串联构成。除输出 32 位求和结果外，还要根据实验要求生成四个标志位：

- 进位标志 CF
- 符号标志 SF
- 溢出标志 OF
- 零标志 ZF

其基本定义如下：

$$SF = F_{31}$$

$$ZF = \overline{F_{31} + F_{30} + \cdots + F_0}$$

对于补码有符号加法，溢出标志可以由最高位相关信号构造；进位标志由加法器高位进位输出得到。通过这些标志位，32 位加法器不仅能够完成普通加法，还可以为后续 ALU 的比较、分支判断等功能提供支持。

3.3.2 电路设计

在主电路中放置两个 16 位加法器、逻辑门、分线器等部件，并在输出端构建标志位生成逻辑。

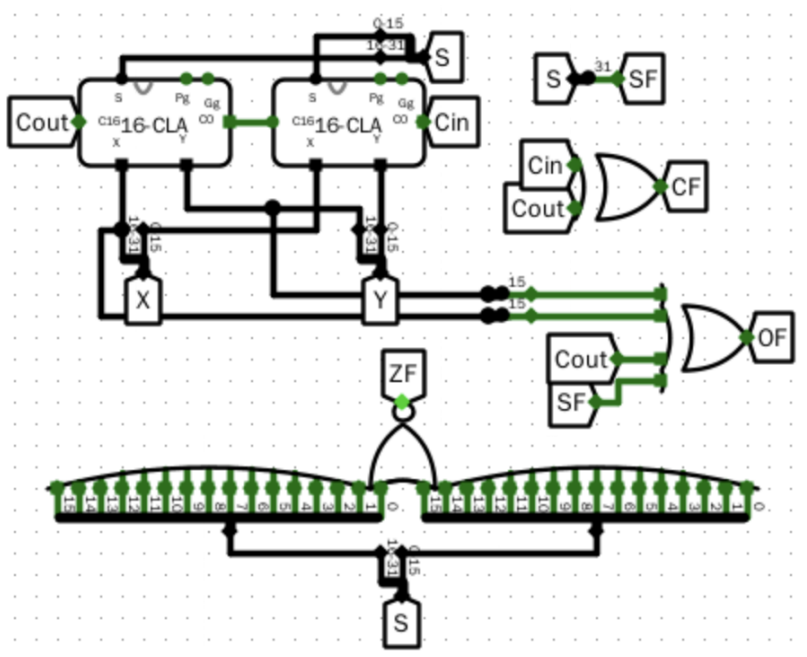


图 9: 32 位快速加法器电路图

3.3.3 仿真测试与分析

输入不同的 32 位操作数进行测试, 观察结果输出及各标志位变化。测试结果表明, 32 位快速加法器可正确完成加法运算, 并能正确生成 CF、SF、OF 和 ZF。

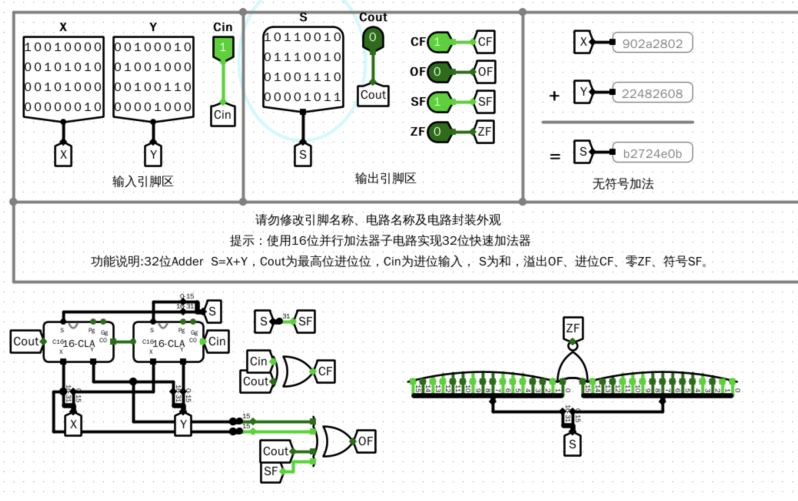


图 10: 32 位快速加法器仿真测试图

3.4 Lab4.4: 32 位桶形移位器

3.4.1 实验原理

32 位桶形移位器是在实验二 8 位桶形移位器的基础上扩展而来。由于 32 位数据最多可移动 31 位, 因此移位量需要 5 位二进制数表示, 即:

$$shamt[4:0]$$

整个 32 位桶形移位器通常可划分为 5 级结构:

- 第 1 级控制是否移动 1 位;
- 第 2 级控制是否移动 2 位;
- 第 3 级控制是否移动 4 位;
- 第 4 级控制是否移动 8 位;
- 第 5 级控制是否移动 16 位。

实验中, 控制信号主要包括:

- L/R : 控制左移或右移;
- A/L : 控制算术移位或逻辑移位;
- $shamt$: 控制移位位数。

3.4.2 电路设计

根据实验要求, 在 Logisim 中扩展得到 32 位桶形移位器, 并完成封装, 便于在 ALU 中复用。

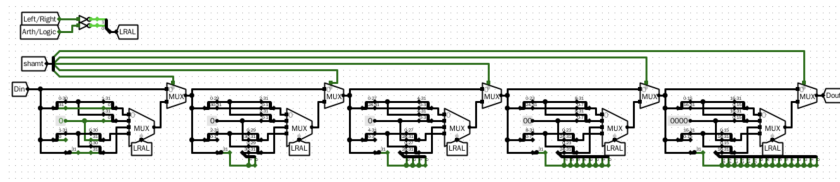


图 11: 32 位桶形移位器电路图

3.4.3 仿真测试与分析

通过设置不同输入数据和控制信号, 对逻辑左移、逻辑右移和算术右移等功能进行测试。仿真结果表明, 32 位桶形移位器可以根据控制信号完成不同方向 and 不同类型的移位操作, 输出结果与预期一致。

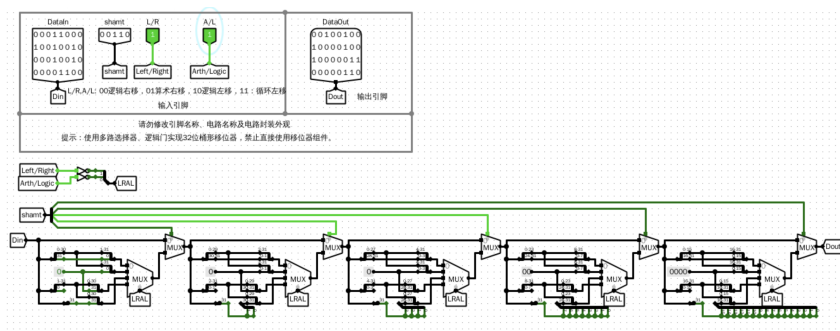


图 12: 32 位桶形移位器仿真测试图

3.5 Lab4.5: RV32I 算术逻辑部件 ALU

3.5.1 实验原理

ALU 是 CPU 中的核心运算部件之一, 用于完成算术运算、逻辑运算、移位运算和比较运算。根据实验要求, 本实验设计的 RV32I ALU 包含如下几类功能:

- 算术运算: 加法、减法;
- 逻辑运算: 与、或、异或;
- 移位运算: 逻辑左移、逻辑右移、算术右移;
- 比较运算: 带符号小于比较、无符号小于比较;
- 直接输出操作数 B。

ALU 的功能由 4 位控制信号 $ALUctr$ 决定。控制器根据输入的 $ALUctr$ 生成各内部控制信号，例如：

- $SUBctr$ ：控制加/减法；
- $SIGctr$ ：控制有符号/无符号比较；
- $LRctr$ ：控制左移/右移；
- $ALctr$ ：控制逻辑移位/算术移位；
- $OPctr[2:0]$ ：控制输出结果的多路选择。

因此，本实验分为两个部分：一是设计 ALU 控制部件 $ALUctrl$ ，二是将 32 位加法器、32 位移位器、逻辑门阵列和控制部件组合成完整的 RV32I ALU。

3.5.2 $ALUctrl$ 电路设计

根据实验给出的控制逻辑表达式，在 Logisim 中设计 ALU 操作控制部件，使其能够根据 $ALUctr$ 输出内部控制信号。

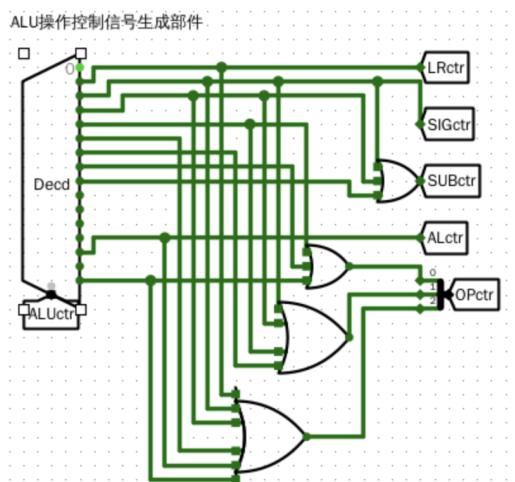


图 13: $ALUctrl$ 电路图

3.5.3 $ALUctrl$ 仿真测试

改变输入控制码 $ALUctr$ ，观察各内部控制信号输出是否符合实验要求，从而验证控制部件逻辑正确。

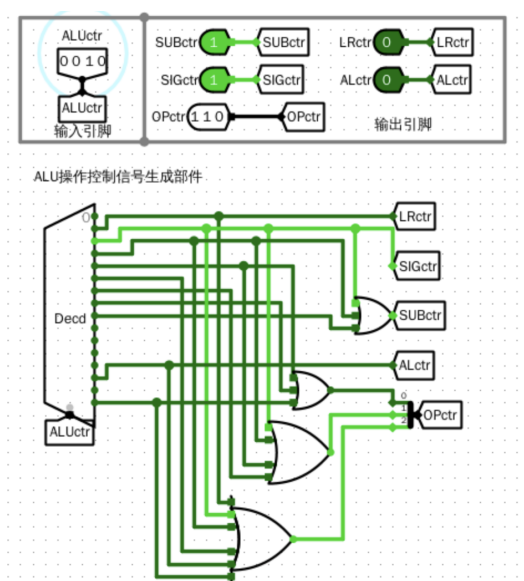


图 14: ALUctrl 仿真测试图

3.5.4 RV32I ALU 电路设计

在主电路中放置 32 位加法器、32 位桶形移位器、ALUctrl、逻辑门阵列、多路选择器、扩展器等模块，按实验原理图完成连接，构建出完整的 RV32I ALU。

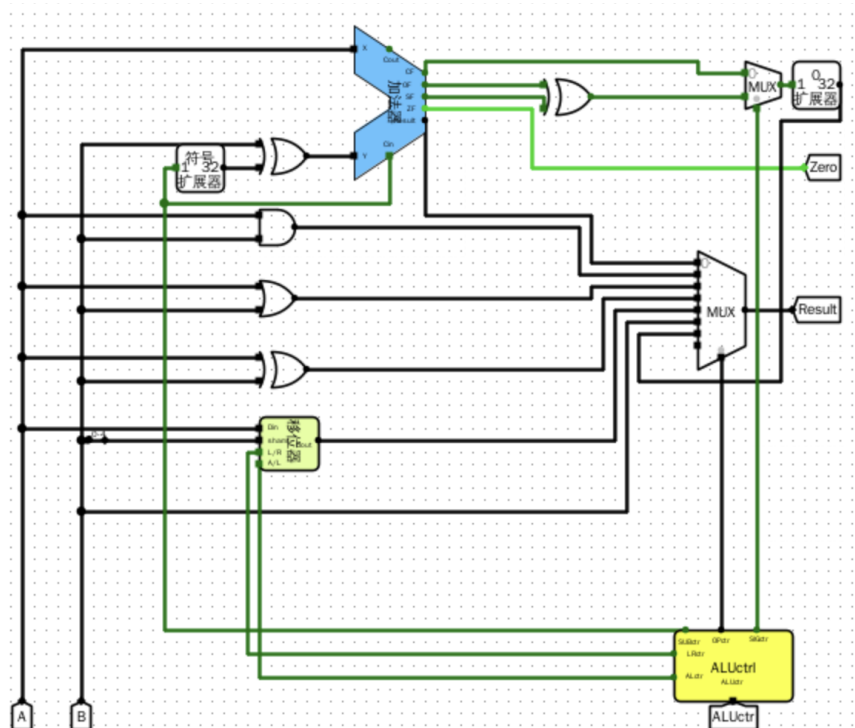


图 15: RV32I ALU 电路图

3.5.5 RV32I ALU 仿真测试

按照实验给出的 ALUctr 编码方式, 对不同操作数和不同控制码组合进行测试, 验证 ALU 各类运算功能是否正确, 包括加法、减法、逻辑运算、移位运算、比较运算以及结果为零标志 Zero 的生成。

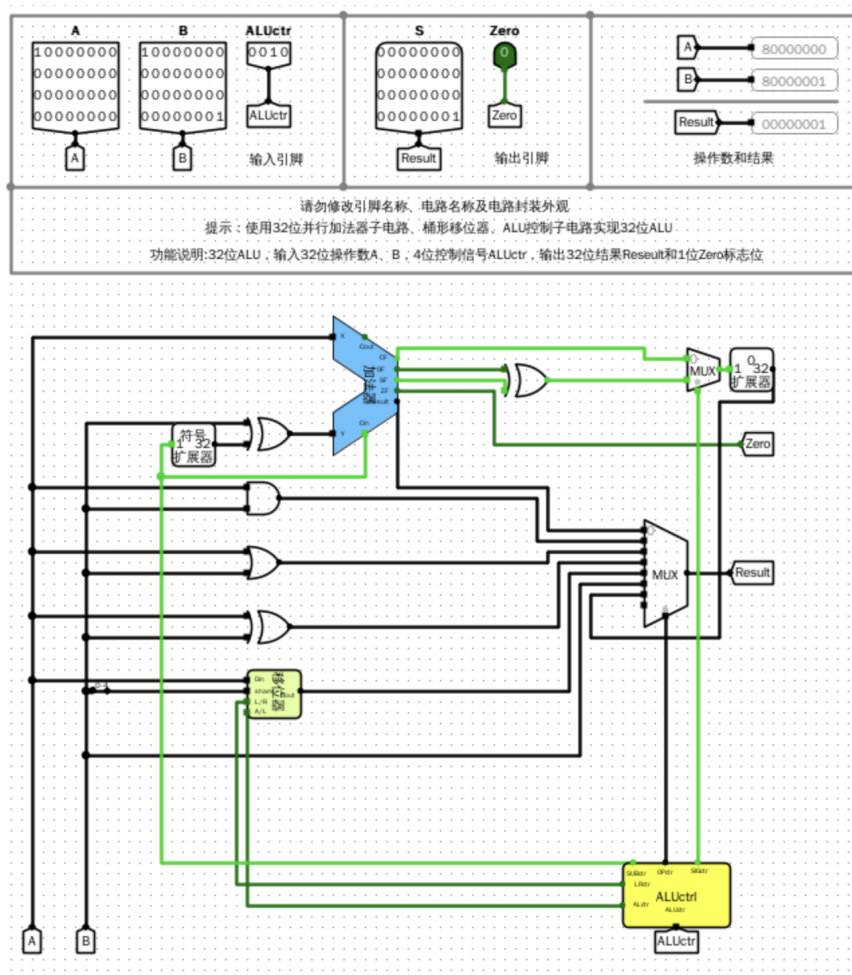


图 16: RV32I ALU 仿真测试图

3.5.6 错误分析

在本实验的 ALU 设计过程中, 主要错误出现在扩展器的设置上。最初连接比较结果相关通路时, 扩展器被误设置为**符号扩展**, 导致高位自动补符号位, 使得部分本应输出为逻辑意义上的 0/1 比较结果在扩展到 32 位后出现异常, 进而影响了最终 Result 的正确性。

具体表现为: 当比较结果仅需要将低位的 1 位结果扩展为 32 位数据输出时, 若采用符号扩展, 则当最高有效位被当作符号位处理后, 高位会被补成全 1, 这与实验中“小于置位”结果应当只在最低位体现、其余高位补 0 的要求不符。因此在仿真中会出现输出结果异常的情况。

排查后发现问题根源在于扩展器模式选择错误。将扩展器由**符号扩展**修改为**0 扩展**后，高位统一补 0，比较结果恢复正常，ALU 各项功能测试也能够正确通过。

对应的错误测试现象如下图所示：

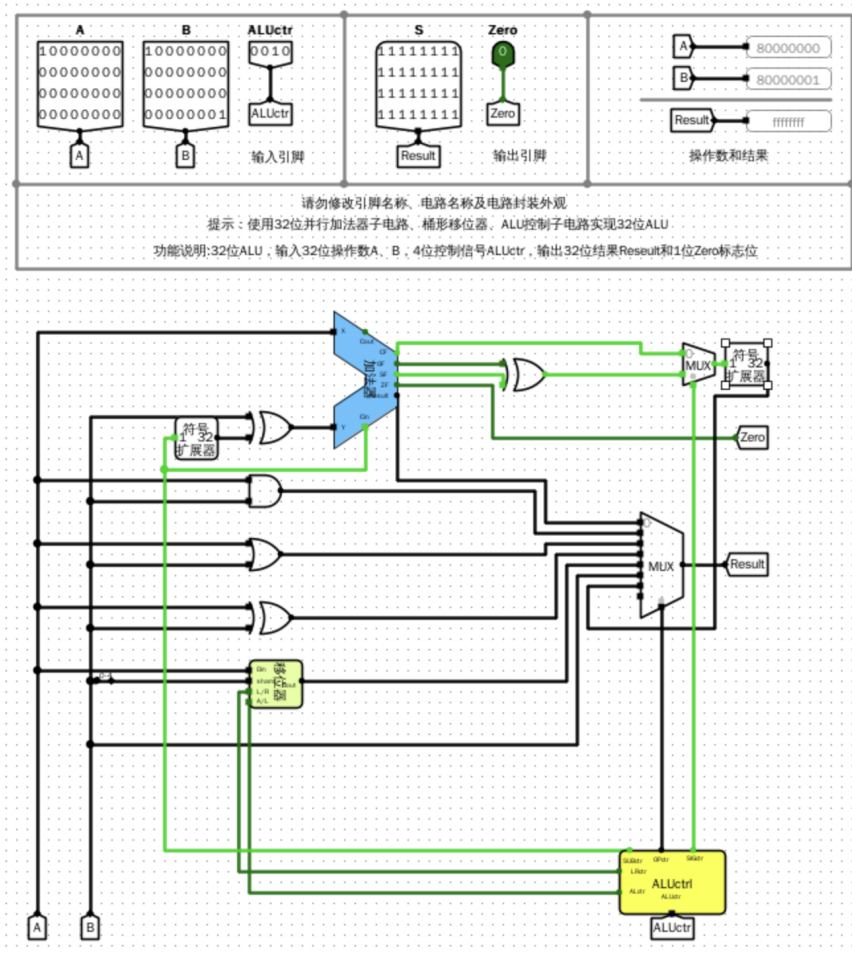


图 17: RV32I ALU 错误测试图

4 实验总结

通过本次实验，我完成了先行进位部件 CLU、4 位快速加法器、16 位两级先行进位加法器、32 位快速加法器、32 位桶形移位器以及 RV32I 算术逻辑部件 ALU 的设计与验证。实验过程中，我进一步掌握了以下内容：

1. 理解先行进位思想及其在快速加法器设计中的作用；
2. 掌握分组先行进位结构，从低位 CLA 扩展到高位快速加法器；
3. 学会在加法器基础上设计并生成多种标志位；
4. 掌握 32 位桶形移位器的分级结构与控制方法；

5. 理解 RV32I ALU 的整体组成以及控制信号生成方式;
6. 学会通过模块封装和层次化设计构建较复杂的数据通路部件。

本次实验将加法器、移位器、逻辑运算阵列和控制逻辑综合起来, 实现了较完整的 RV32I 运算部件设计。通过本次实验, 我对 CPU 核心数据通路中的算术逻辑部件有了更加系统的理解。

5 思考题

5.1 1. 通过查找资料, 实现一种 64 位二进制快速加法器的设计

实现 64 位二进制快速加法器的一种可行方法是采用**分组先行进位结构**。具体来说, 可以将 64 位加法器划分为若干个较小的 CLA 分组, 例如划分为 16 个 4 位 CLA, 或者 4 个 16 位加法器模块, 再通过更高层的先行进位部件统一生成组间进位。

其基本思想与本实验的 16 位、32 位快速加法器一致, 即:

- 组内使用先行进位方式减少位间依赖;
- 组间再使用更高一级的先行进位逻辑;
- 通过层次化设计降低关键路径延迟。

若继续追求更高速度, 还可以采用前缀加法器结构, 如 Kogge-Stone、Brent-Kung 或 Carry Select Adder 等形式。但在 Logisim 实验环境下, 分组 CLA 结构更容易实现, 也更便于调试与验证。

本题中, 我采用了由多个低位 CLA 模块级联扩展的方法来实现 64 位快速加法器。电路结构延续了实验中 32 位快速加法器的设计思路, 具有较好的模块化和可扩展性。

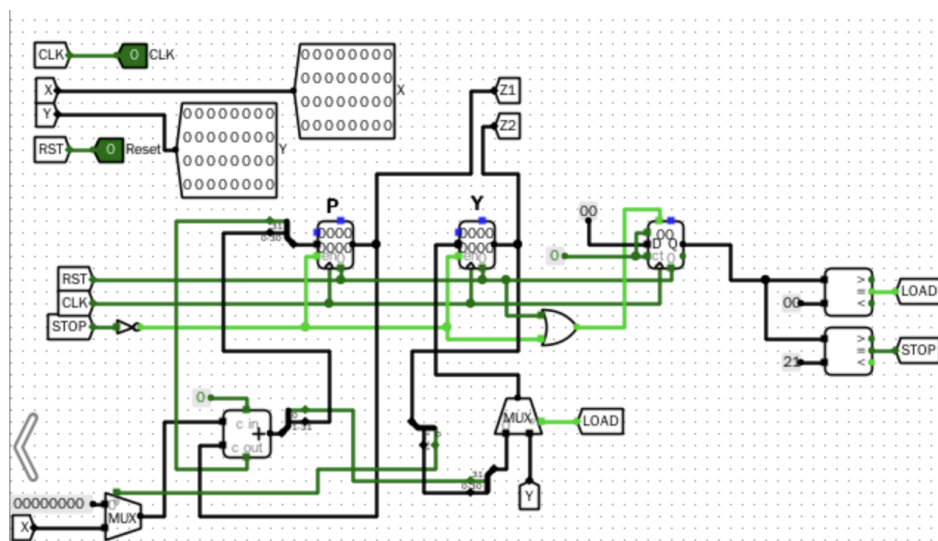


图 18: 思考题 1: 64 位快速加法器设计

5.2 2. 将实验 3 中的快速乘法器扩展到 32 位无符号数相乘, 并探讨如何将该乘法器融合到实验中的 ALU 电路来实现乘法运算

将实验 3 中的快速乘法器扩展到 32 位无符号数相乘时, 需要考虑如下问题:

- 输入位宽从 4 位扩展到 32 位;
- 中间积寄存器和结果寄存器位宽需要扩展到 64 位;
- 控制计数器至少需要能够完成 32 次移位与累加控制;
- 加法器、移位器和寄存器位宽都要同步扩展。

若要将乘法器融合到 ALU 中实现乘法运算, 可以采用如下方式:

- 1) 在 ALU 中新增一个乘法器子电路;
- 2) 在 ALU 控制器中加入新的控制编码, 使 ALU_{ctr} 能选择乘法运算;
- 3) 在结果多路选择器中增加一个输入端, 用于选择乘法器输出结果;
- 4) 若只保留低 32 位结果, 则直接输出乘积低 32 位; 若需完整结果, 则应扩展结果输出总线位宽。

这种扩展方式可以增强 ALU 的运算能力, 但也会增加硬件复杂度和关键路径长度, 因此在实际处理器设计中, 乘法器通常作为独立执行单元存在。

本题中, 我采用了将实验 3 的乘法器结构扩展到 32 位的方法, 并保留了“加法 + 移位 + 计数控制”的基本实现思路, 使其能够输出更高位宽的乘法结果。

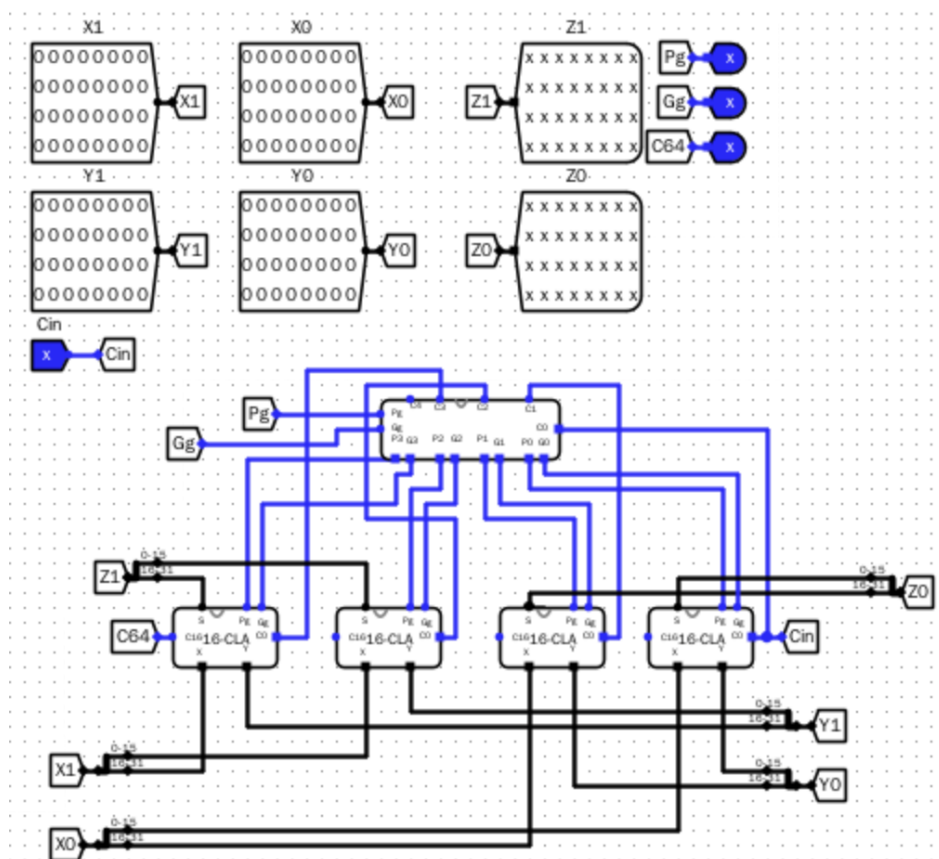


图 19: 思考题 2: 32 位无符号快速乘法器设计

5.3 3. 假设在 RV32I 中新增一条指令，导致在 ALU 中增加一个新运算操作，试修改 ALU 设计电路，并通过测试数据进行验证

若在 RV32I 中新增一条运算指令，则可以按以下步骤修改 ALU：

- 1) 为新运算分配一个新的 ALU_{ctr} 编码；
- 2) 在 ALU_{ctrl} 中增加相应控制信号输出逻辑；
- 3) 在数据通路中加入对应的新运算模块，或利用现有模块重新组合；
- 4) 扩展结果多路选择器的输入端数，使其能够选择该新运算结果；
- 5) 设计测试用例验证功能是否正确。

本题中，我新增的运算直接采用了思考题 2 中实现的乘法运算，即在原有 ALU 的基础上加入乘法器模块，并为其分配新的控制编码。当控制信号选择该功能时，ALU 输出乘法器的运算结果，从而实现对乘法指令的支持。

这种修改方式说明，ALU 的模块化设计不仅可以支持加减、逻辑、移位和比较等基本操作，还可以较方便地扩展新的复杂运算。通过增加新的结果选择通路和控制逻辑，可以逐步增强 ALU 的功能，为后续更完整的处理器设计打下基础。

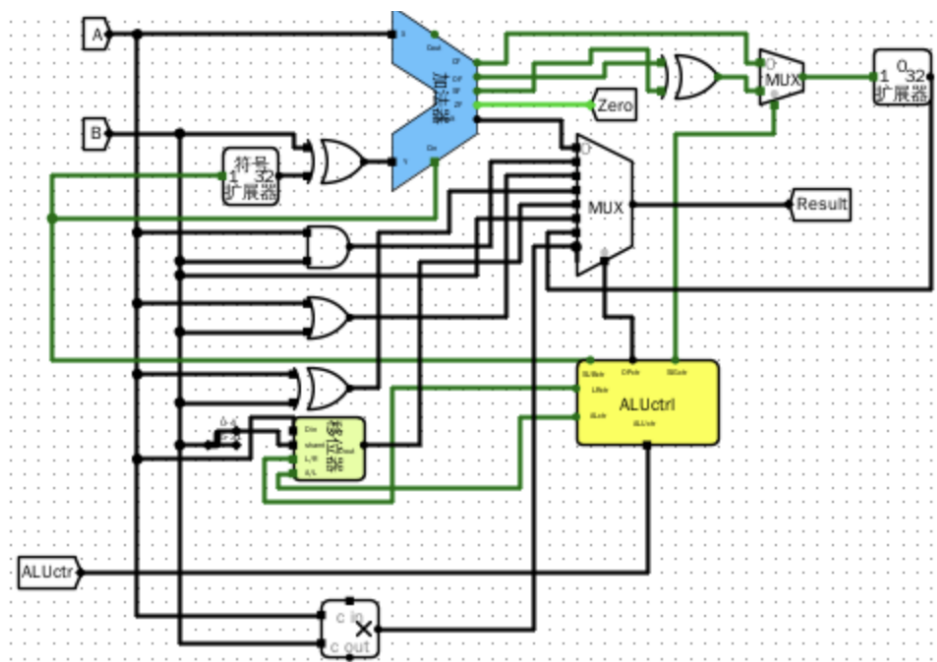


图 20: 思考题 3: 在 ALU 中加入乘法运算后的电路设计

5.4 4. 分析比较运算使用独立的比较器和使用减法运算通过标志位来实现两种方法的特性

比较运算有两种常见实现方法。

(1) **使用独立比较器** 独立比较器直接根据两个操作数的各位关系生成比较结果。其特点是：

- 结构直观，逻辑功能明确；
- 对专门的比较操作而言，延迟较稳定；
- 但需要额外设计专门的比较硬件，增加电路资源开销。

(2) 使用减法运算结合标志位 另一种方法是先计算:

$$A - B$$

再根据 Zero、Sign、Overflow、Carry 等标志位判断 A 与 B 的关系。其特点是：

- 可以复用已有加法器硬件，节省资源；
- 易于与 ALU 中的减法运算共享数据通路；
- 适合处理器中统一实现比较与分支判断；

- 但判定逻辑相对复杂，需要结合有符号/无符号情况分析不同标志位组合。

综合来看，在处理器 ALU 中通常更倾向于采用“减法 + 标志位”的实现方式，因为这样可以减少专用比较硬件，提升整体资源利用率；而在对比较速度要求较高或逻辑较独立的场景下，独立比较器也有其优势。